Reviewer Pack — Fourier Coefficient Decay Bounds AC⁰ Circuit Size for Parity…

Entry 29d6f7d21e9f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 05:43:17 UTC

Fourier Coefficient Decay Bounds AC⁰ Circuit Size for Parity-Insensitive Functions

Entry ID: `29d6f7d21e9f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 05:43:17 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis on Finite Groups **Field B** (complexity object): AC⁰ Circuit Size

Statement:

> For any parity-insensitive Boolean function f: {0,1}^n → {0,1}, the sum of absolute Fourier coefficients ∑_{S≠□} |f̂(S)| decays exponentially with n iff the minimal AC⁰ circuit size computing f is Ω(2^{n/2}).

Rationale (proposer's reasoning):

> Parity-insensitive functions exhibit Fourier coefficient concentration patterns that may correlate with algebraic complexity. The exponential decay threshold could expose a structural gap between Fourier analytic simplicity and circuit complexity, leveraging the inherent symmetry of AC⁰ computation.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `581992b396f983b1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def fast_walsh_hadamard_transform(f):
    n = len(f)
    while n > 1:
        for i in range(n // 2):
            for j in range(i, i + n // 2):
                temp = f[j]
                f[j] += f[j + n // 2]
                f[j + n // 2] = temp - f[j + n // 2]
        n //= 2
    return f

def compute_fourier_coefficients(clauses, n):
    f = [0] * (1 << n)
    for clause in clauses:
        product = 1
        for literal in clause:
            if literal > 0:
                product *= 1 - 2 * f[literal - 1]
            else:
                product *= 1 + 2 * f[-literal - 1]
        f[0] += product
    return [abs(coeff / (1 << n)) for coeff in fast_walsh_hadamard_transform(f)]

def generate_3sat_clauses(n, m):
    clauses = []
    variables = list(range(1, n + 1))
    for _ in range(m):
        clause = random.sample(variables, random.randint(1, n))
        if random.choice([True, False]):
            clause = [-x for x in clause]
        clauses.append(clause)
    return clauses

def compute_decision_tree_depth(circuit):
    depth = 0
```

```
    queue = [circuit]
    while queue:
        next_level = []
        for node in queue:
            if isinstance(node, list):
                next_level.extend(node)
        queue = next_level
        depth += 1
    return depth

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    m = 2 * n
    clauses = generate_3sat_clauses(n, m)
    f_hat = compute_fourier_coefficients(clauses, n)
    sum_abs_fourier_coeffs = sum(f_hat[1:])

    # Simulate AC⁰ circuit size (simplified model)
    ac0_circuit_size = 2 ** (n // 2)

    decision_tree_depth = compute_decision_tree_depth(circuit)
    if decision_tree_depth > ac0_circuit_size:
        return {
            "metric_name": "sum_abs_fourier_coeffs",
            "metric_value": sum_abs_fourier_coeffs,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Decision tree depth is too large for the circuit size"
        }

    return {
        "metric_name": "sum_abs_fourier_coeffs",
        "metric_value": sum_abs_fourier_coeffs,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
```

```
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Decision tree depth exceeds circuit size\" firs
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b03edf7.py", line 97, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b03edf7.py", line 67, in run_trial
    f_hat = compute_fourier_coefficients(clauses, n)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b03edf7.py", line 38, in compute_fouri
    return [abs(coeff / (1 << n)) for coeff in fast_walsh_hadamard_transform(f)]
                                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b03edf7.py", line 23, in fast_walsh_ha
    f[j] += f[j + n // 2]
            ~^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with IndexError before producing results, preventing evaluation of conjecture validity | next: Debug fast_walsh_hadamard_transform implementation to fix index out of range error

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	84629
2	preregistration	ollama_remote	qwen3:8b	0	0	24136
3	novelty	ollama_remote	qwen3:8b	0	0	20640
4	novelty	ollama_remote	qwen3:8b	0	0	15776
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13687
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10505

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	judge	ollama_remote	qwen3:8b	0	0	15066

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 184439 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/29d6f7d21e9f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/29d6f7d21e9f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/29d6f7d21e9f.tar.gz` (if generated)